

PRACTICAL 5

1. Title: Apply array methods and object handling; create a cart total calculator with discount logic.

2. Software/Tools Required: Visual Studio Code (VS Code), JavaScript.

3. Theory:

JavaScript objects allow storing collections of data using key-value pairs (e.g., name, price, quantity), while arrays store ordered lists of these objects. Higher-order array methods

such as `reduce()` process elements systematically without standard explicit loops. The `reduce()` method executes a reducer function on each array element, accumulating values to

compute a single output—such as the total cost of all products.

Conditional statements (if-else or ternary operators) evaluate business logic, such as applying a 10% discount if the cart's total exceeds a specific threshold.

4. Output:

```
main.js Run ▶
9  },
10 {
11     name: "Mouse",
12     price: 1000,
13     quantity: 2
14 },
15 {
16     name: "Keyboard",
17     price: 2000,
18     quantity: 1
19 },
20 {
21     name: "Headphones",
22     price: 3000,
23     quantity: 1
24 }
25 ];
26
27 console.log("==== PRANATI'S SHOPPING CART =====");
28
29 // Display all products
30 cart.forEach(item => {
31     console.log(
32         `${item.name} - ₹${item.price} x ${item.quantity}`
33     );
34 });
```

Output

```
==== PRANATI'S SHOPPING CART =====
Laptop - ₹50000 x 1
Mouse - ₹1000 x 2
Keyboard - ₹2000 x 1
Headphones - ₹3000 x 1

Item Totals:
[ 50000, 2000, 2000, 3000 ]

Subtotal: ₹57000
Discount: ₹11400
Final Cart Total: ₹45600

Products costing ₹3000 or more:
Laptop - ₹50000
==== PRANATI'S prn : 24070521178 =====
Headphones - ₹3000
==== PRANATI'S prn : 24070521178 =====

=== Code Execution Successful ===
```

5. Result/Conclusion: The experiment was performed successfully. JavaScript arrays, objects, and higher-order array methods (specifically `reduce()`) were demonstrated to process structured data efficiently. Conditional logic for applying discounts was implemented, and the final output was verified using `console.log()` methods.